

ROS and experimental robotics. Part 3: project.

1 Context, goal and evaluation of the project

In this project, you will control a simulated and a real Turtlebot 3 burger mobile robot in a realistic challenging environment, so as to make it navigate from one starting position to a goal position with different tasks to solve on the path. The navigation should successively exploit:

- images obtained from a simulated/real camera to detect and follow some lines,
- a laser scan obtained from a simulated/real LDS to detected and avoid some obstacles,
- and finally both of them to navigate in a challenging environment where both sensors are required together.

The project will take the form of 5 successive challenges relying on the sensorimotor capabilities of the turtlebot, as illustrated in Figure 1.

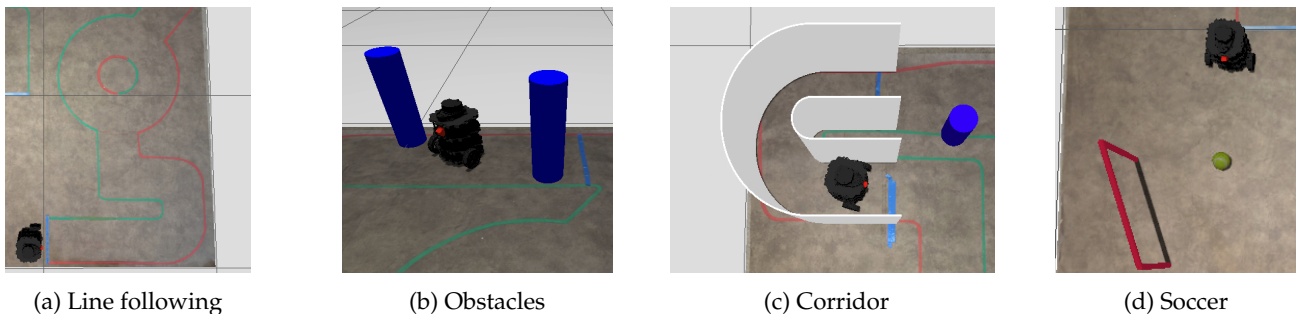


Figure 1: Illustration of the Gazebo world you will work on.

1.1 Description of the organization of your work

All your developments (code, launchfiles, scripts, gazebo worlds, configuration files, etc.) must be placed inside one and unique ROS package named `projet`. You can find on Moodle the `projet2025` package, including the simulated world we provide for the project (see Figure 2), together with the corresponding launchfile spawning a turtlebot 3 burger endowed with a LDS and camera sensors. Of course, these files only serve as a basis for your requirements, and you are free to modify them accordingly.

#1.1.1. Unzip the `projet2025` package in your ROS2 workspace (e.g., `~/ros2_ws/src/`), then build the workspace using:

```
cd ~/ros2_ws
colcon build --packages-select projet2025
source ~/.bashrc
```

Congratulations, you are now ready to work on the project!

1.2 Evaluation

At the end of the project, you will have to:

- Submit a compressed archive of your `projet` package via Moodle;
- Give a 7-minute presentation of your work 7min ($\pm$ 7-8 slides) , followed by a 7-minute Q&A session; The presentation should focus on explaining your system architecture and the implemented solutions, **without showing code**. You are expected to clearly illustrate how your algorithm works, using **algorigrams** (you can use tools such as <https://app.diagrams.net/>) and illustrations of the image or signal analysis. You must also explain how the work was divided within the group and who was responsible for each part. Please note that the quality of your presentation will also be evaluated. We expect you to be concise and clear in your explanations. Any failure to do so will result in a lower grade. Indeed, a lack of perspective or synthesis will be interpreted as a lack of understanding of the work you are presenting.

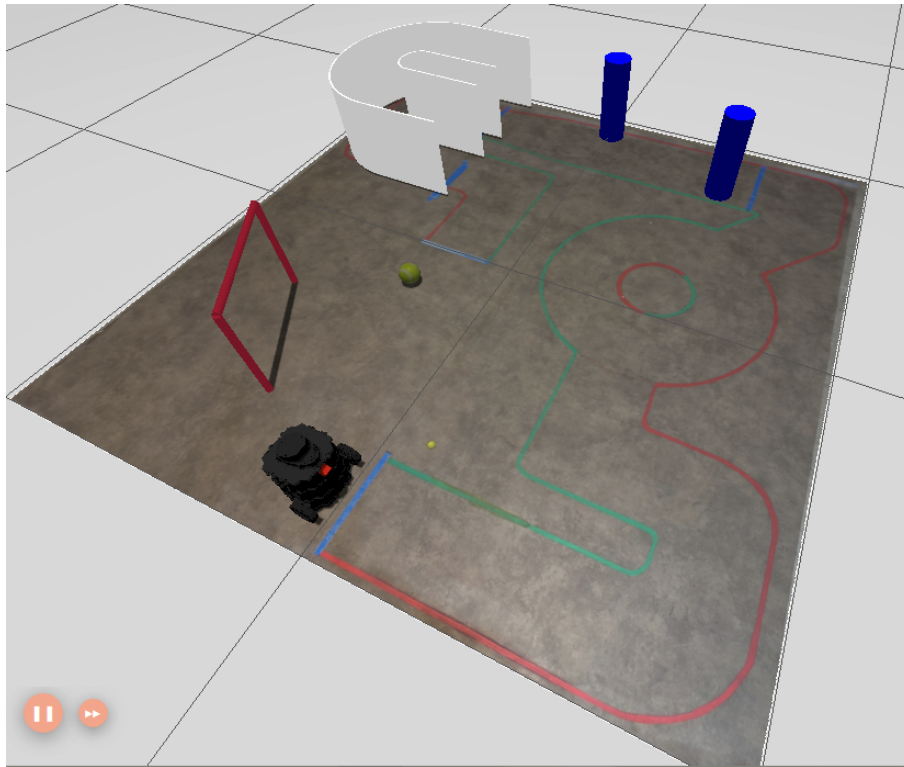


Figure 2: The entire gazebo world, with the 4 challenges to solve: (1) line following, (2) obstacle avoidance, (3) movement in a corridor, and (4) score a goal

- Demonstrate your work on the real robot. **Only the results successfully demonstrated on the robot during your evaluation slot will be considered for grading.** This means your code must be fully operational on the day of the exam and at the time of your scheduled session. You will have approximately 30 minutes in the lab with the evaluator to set up and run your demo.

Evaluation

- **/100 points:** Presentation of the coded solution — quality of the presentation, accuracy of ROS-related vocabulary, adherence to formatting guidelines, and clarity of responses to questions.
- **/170 points:** Demonstration — evaluation based solely on concrete results, showcasing the robot's actual capabilities (detailed in each section).

1.2.1 Organization

You are free to create as many ROS2 nodes as needed and to organize your package as you see fit. However, we strongly encourage you to make full use of the tools introduced during the course, such as launch files, parameters, RViz, RQT, and rosbag. These tools will not only help structure your project more efficiently, but also simplify debugging, testing, and visualization of your system.

Practically, we propose the following organization:

- you can work by two, regardless of your initial group. No more than 1 group of 3 students per specialty (SI, MeDH, ROB) is allowed if necessary;
- you have now entire days of practicals;
- for your organization, and because the real robot might not be always available, use extensively `rosbag` to record and replay real data so as to design your algorithms.

Only one ROS package must then be uploaded to Moodle per student group. Do not forget to write down your two names inside some kind of `README` file at the root of your package!

It is important to understand that the "simulation" part of the project must be seen as a way to test, evaluate, assess strategies that should then be implemented on the real robot. This transition from simulation to reality is one of the key point in ROS architectures, and its versality should help you in the process, providing you develop everything in a generic way (with parameters for instance). Again, you can also benefit from recordings, **rosbag**, etc. to develop offline algorithms that should work directly on the real robot, with real-time sensor data. The Gazebo simulation is a tool, and feel free to develop it further if you find it useful (e.g., by adding noise to the camera, etc.).

2 Challenge 1: line following

For this first challenge, you are expected to develop one or more nodes that work together to ensure the TurtleBot3 stays between the two lines marking the path in the environment. However, this challenge is not limited to image processing: at the beginning of the demonstration, the evaluator will ask you to make the robot take the roundabout either on the left or on the right. You must be able to configure this behavior quickly before the run. Additionally, the robot must be able to detect obstacles in its path at all times during the line-following task and react accordingly — for instance, by stopping or adapting its trajectory to avoid a collision.

Throughout this project, you must use the `OpenCV` library to process images published either by Gazebo or by the real robot on the corresponding ROS2 topic. In ROS2, image data is typically received as messages of type `sensor_msgs/msg/Image`, and should be converted to OpenCV format using the `cv_bridge` package.

You can refer to the following resources for help with OpenCV in Python:

- https://docs.opencv.org/master/d6/d00/tutorial_py_root.html
- <https://docs.opencv.org/4.2.0/pages.html>

A package named `my_cv_package` with a minimal example of a ROS2 node (in Python) that subscribes to an image topic, converts the received messages to OpenCV format, and displays them in a window is given on moodle. For this first challenge, you will mainly have to:

- build a new node which subscribes to the image topic published by gazebo or the real robot;
- use basic `OpenCV` functions to process the image. Since you are trying to detect colored lines in the image, a simple binarization of the image, followed by an image moment computation (see https://en.wikipedia.org/wiki/Image_moment) might be an interesting step to extract some simple information you can use to drive the robot from the image, see Listing ?? on page ?? ;
- finally, use this information to actually send a velocity command to the robot.
- In addition, you must implement two distinct strategies that allow the robot to take the roundabout either from the left or from the right. One of these strategies should be selected and activated at the beginning of your demonstration.

Evaluation

- **10/40 points** — The robot is able to follow the lines up to the roundabout, including curved sections, while staying correctly between the two lines.
- **20/40 points** — In addition to the above, the robot is able to perform an emergency stop when an obstacle suddenly appears in front of it. The obstacle will be introduced by the evaluator during the demonstration. It is not expected that the robot avoids it, but it must stop immediately if the obstacle is detected at close range.
- **30/40 points** — The robot is able to take the roundabout successfully, regardless of the direction.
- **40/40 points** — The robot is able to take the roundabout in the direction that was explicitly requested by the evaluator (left or right), in two different and successive trials.

3 Challenge 2: obstacle avoidance

In this second challenge, your robot must continue to follow the road by staying between the colored lines as accurately as possible. However, unlike in the previous phase, it is now permitted for **a single wheel only** to cross the line—if and only if this is strictly necessary to avoid an obstacle. This exception should be used sparingly and kept to a minimum.

Two obstacles will be placed on the path. The robot is expected to detect them and adapt its trajectory accordingly to bypass them, while still continuing to follow the overall line path.

Unlike in Challenge 1, stopping in front of the obstacle is not a valid strategy — the robot must actively navigate around it to complete the task.

Evaluation

- **10/30 points** — The robot is able to follow the road correctly in the presence of obstacles, without deviating significantly from the path. It detects at least one of the two obstacles and adapts its trajectory to avoid it, while maintaining line-following behavior. If necessary, a single wheel may cross the line, but the robot must not leave the lane entirely.
- **20/30 points** — In addition to the above, the robot successfully detects and avoids both obstacles, and proceeds toward the next challenge. The robot may still cross the line with one wheel if required, but should return to the lane promptly after avoidance.
- **30/30 points** — The robot avoids both obstacles while remaining entirely within the lane (no wheel crosses the line). The avoidance maneuvers are smooth, controlled, and followed by stable line following.

4 Challenge 3: corridor navigation

For this third task, you must implement one or more nodes that enable the robot to autonomously navigate through a corridor formed by two parallel walls. The robot must rely exclusively on LIDAR data to perceive and follow the environment—no camera or image processing is allowed in this challenge.

The objective is for the robot to move safely and smoothly through the corridor by maintaining a central position between the two walls, while progressing toward the exit. The robot should continuously adjust its trajectory based on the distance to both the left and right walls, ensuring collision-free motion throughout the navigation.

Successful completion of this challenge requires stable wall-following behavior, consistent speed, and the ability to reach the end of the corridor without human intervention.

Evaluation

- **15/30 points** — The robot is able to move through the corridor using only LIDAR data, without colliding with the walls. The trajectory may be unstable or off-center, but the robot reaches the exit without external help.
- **30/30 points** — The robot maintains a relatively centered trajectory throughout the corridor. The navigation is stable, and the robot adjusts smoothly to both walls to stay within bounds.

5 Challenge 4: Scoring a goal

In this final challenge, a tennis ball is placed near the entrance of the area in a semi-random but reasonable location. The objective is simple: the robot must push the ball into a goal marked by two colored posts.

You are free to use any available sensor (LIDAR, camera, or a combination of both) to detect, approach, and guide the ball toward the goal. The difficulty of this task lies in localizing the ball accurately, aligning with it, and driving it to the target area without losing control of its trajectory.

The evaluation of this challenge is binary: either the ball successfully passes between the two colored posts, or it doesn't. Partial points will only be awarded in specific situations (see below).

Evaluation

- **10/30 points** — The robot detects the ball and approaches it correctly, but fails to push it significantly or loses it during the process.
- **20/30 points** — The robot pushes the ball in the general direction of the goal but fails to get it fully between the posts.
- **30/30 points** — GOOOOOOOOOOOAAAAL! - The robot successfully pushes the ball between the two posts, scoring a valid goal.

6 Challenge 5: Human Motion Control

In this final challenge, your goal remains the same: push a tennis ball between the two colored posts. However, the robot will no longer be autonomous. You must develop a teleoperation system that allows a pilot to control the robot using body movements captured by a webcam.

You are expected to use a vision-based framework (such as MediaPipe or any other relevant model) to track the pilot's movements and translate them into robot commands. The challenge lies in creating an intuitive mapping between human gestures and the robot's displacement to accurately guide the ball into the goal.

The evaluation of this challenge is binary: either the ball successfully passes between the two colored posts, or it doesn't.

Evaluation

- **10/20 points** — The pilot successfully moves the robot using body gestures and approaches the ball, but fails to push it or loses control.
- **20/20 points** — GOOOOOOOOOOOAAAAL! - The pilot successfully maneuvers the robot to push the ball between the two posts using motion control.

Beyond the individual evaluation of each challenge, additional points may be awarded based on the overall level of system integration and autonomy demonstrated during the final evaluation. In particular, the ability to transition from one challenge to the next without human intervention will be assessed as a separate criterion.

Evaluation

- **0/10 points** — The robot must be physically repositioned by hand between each challenge, and manual launching or stopping of specific nodes is required.
- **5/10 points** — The robot remains in the environment throughout the demonstration, but transitions between challenges require remote intervention (e.g., changing parameters or restarting nodes manually). Minimal automation is in place.
- **10/10 points** — Transitions between challenges are handled autonomously through appropriate design (e.g., using launch files, parameters, internal state machines, or behavior trees). The robot can complete multiple challenges in succession without human input.
- **+10 points** — The number of challenges completed autonomously in a continuous sequence will be considered.